

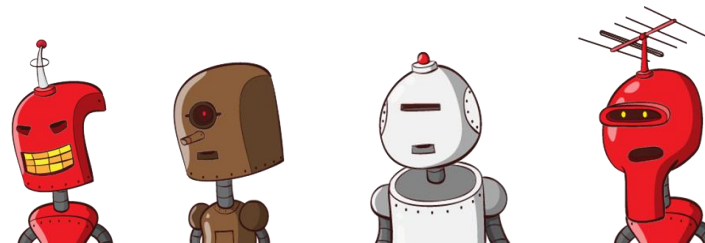
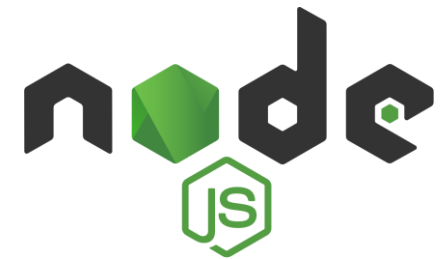
Activity12

Request Methods GET, POST,
DELETE, UPDATE

Next class



Express



Part 1 : In this Lecture we are going to work only with POST.

We are going to inherit the development from Friday's topic:

[12_3_30_Robots_GET_PUT_POST_DELETE.pdf](#)

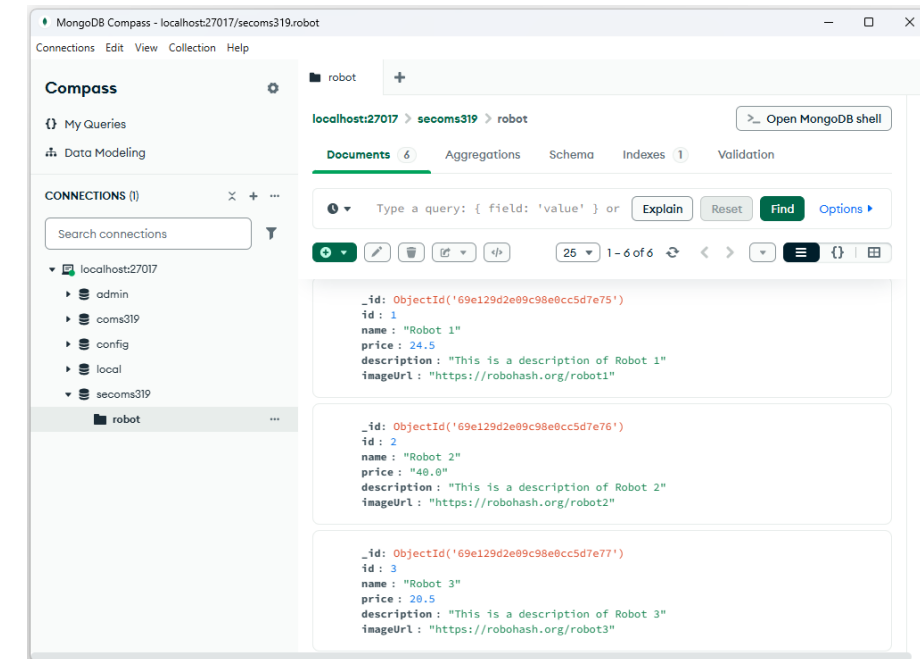
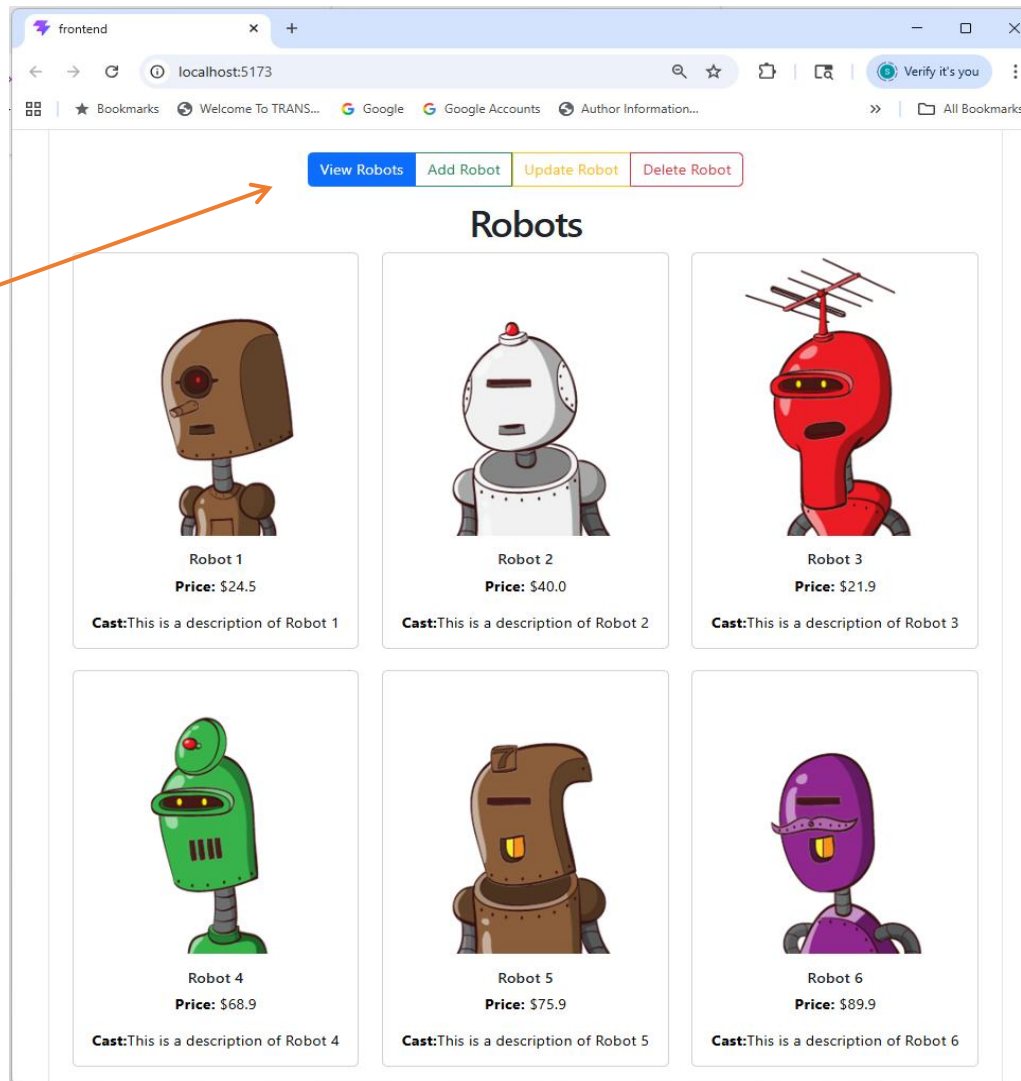
Part 2: The next Lecture, we are going to add the request methods PUT and DELETE.

So far, we can GET data from the collection `robot` in MongoDB.

We can read all robots using : <http://localhost:8081/listRobots>

Or we can read one robot using : <http://localhost:8081/2>

We can add a
buttons for
handling all the
request
methods



Make two directories in your project: Backend & Frontend

Backend

- Create directly without any commands, just press Ctrl + Shift + N or Cmd + Shift + N
- Then follow the steps in the next slide

Frontend

- Create using the React Project Command:
- `npm create vite@latest frontend`
 - i. `cd frontend`
 - ii. `npm install`



Install the Modules:

```
npm install express cors body-parser mongodb nodemon
```

Run nodemon:

```
nodemon index.js
```



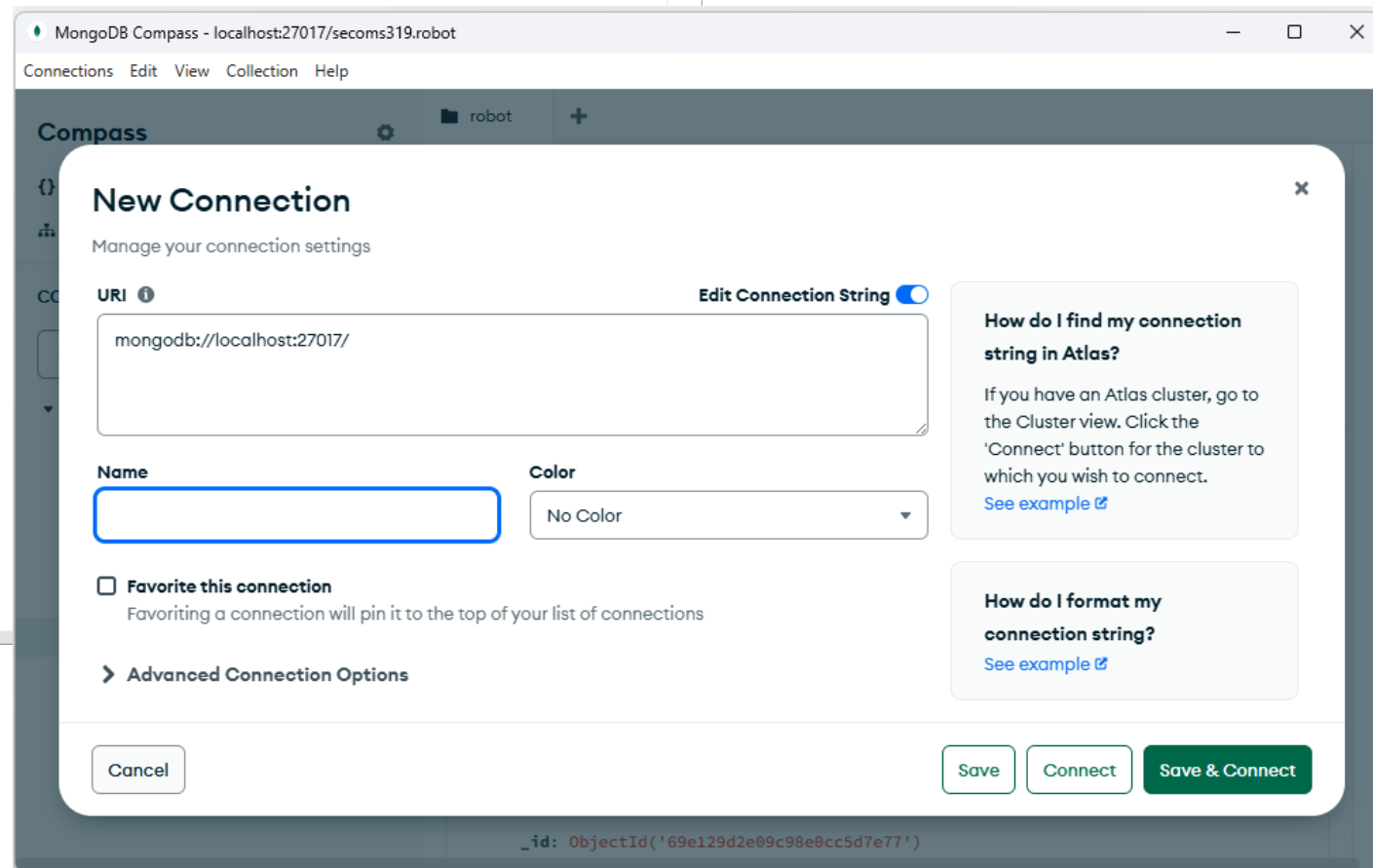
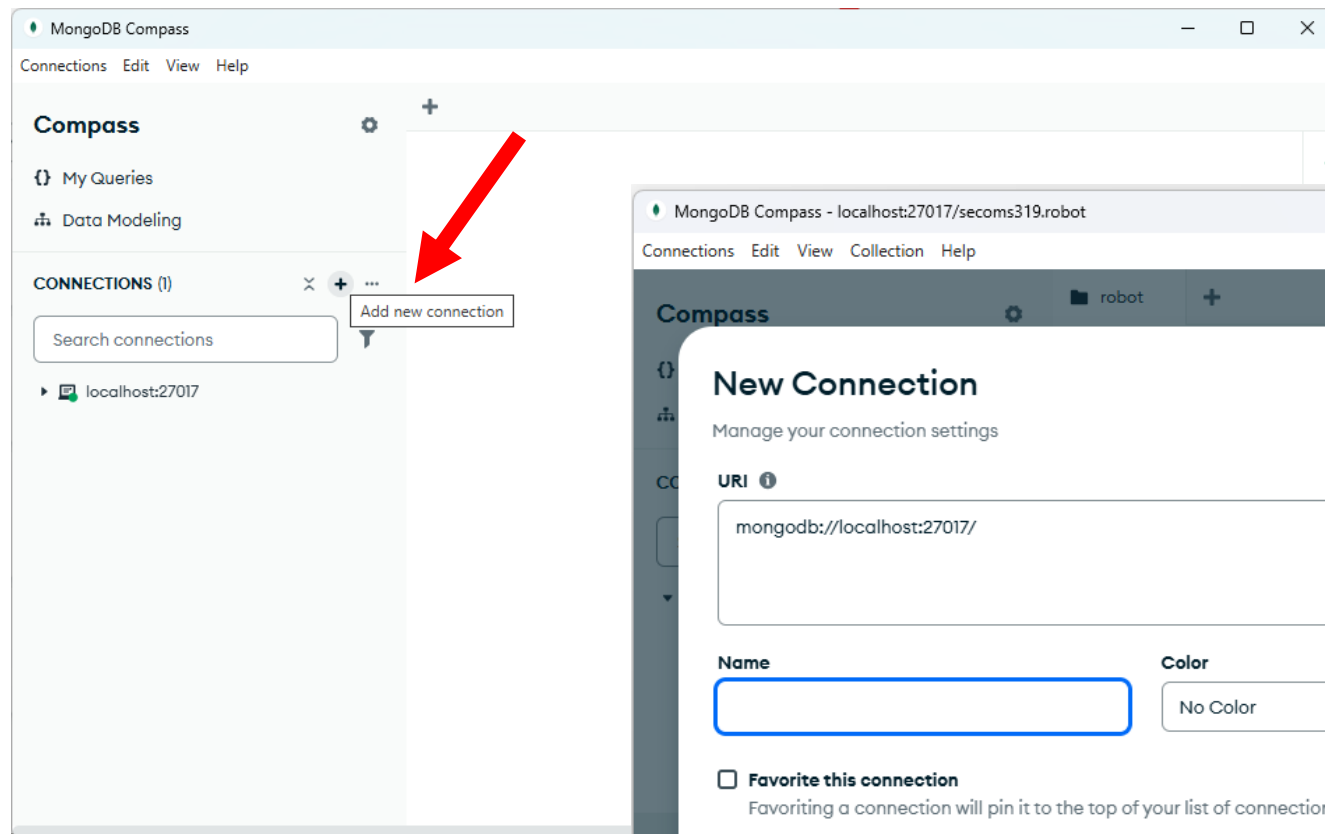
Reference:

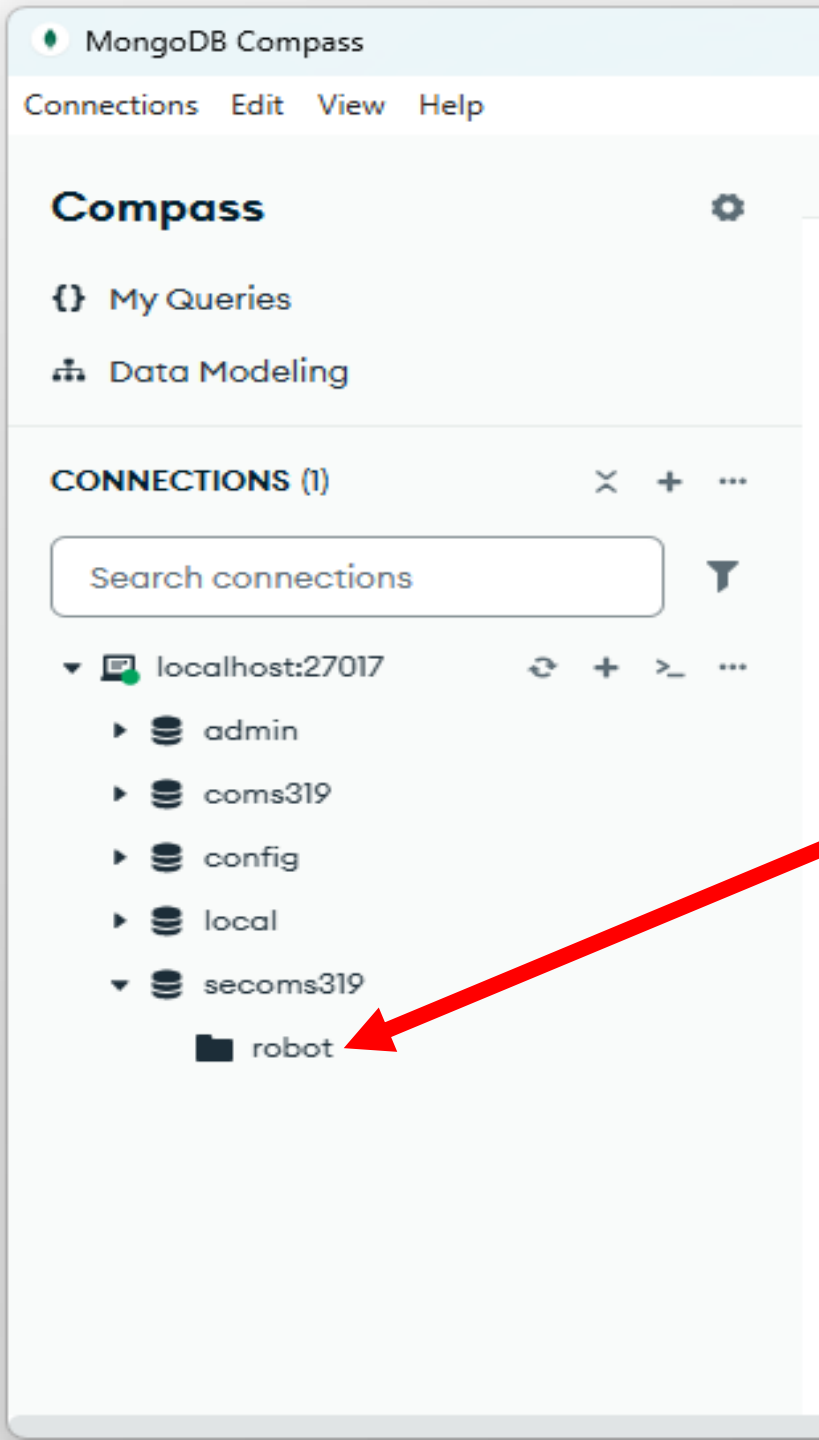
<https://www.mongodb.com/languages/express-mongodb-rest-api-tutorial>

Sarfaraz Ahmed Mohammed, Ph.D.
Ali Jannesari, Ph.D.

Create database **secoms319** :

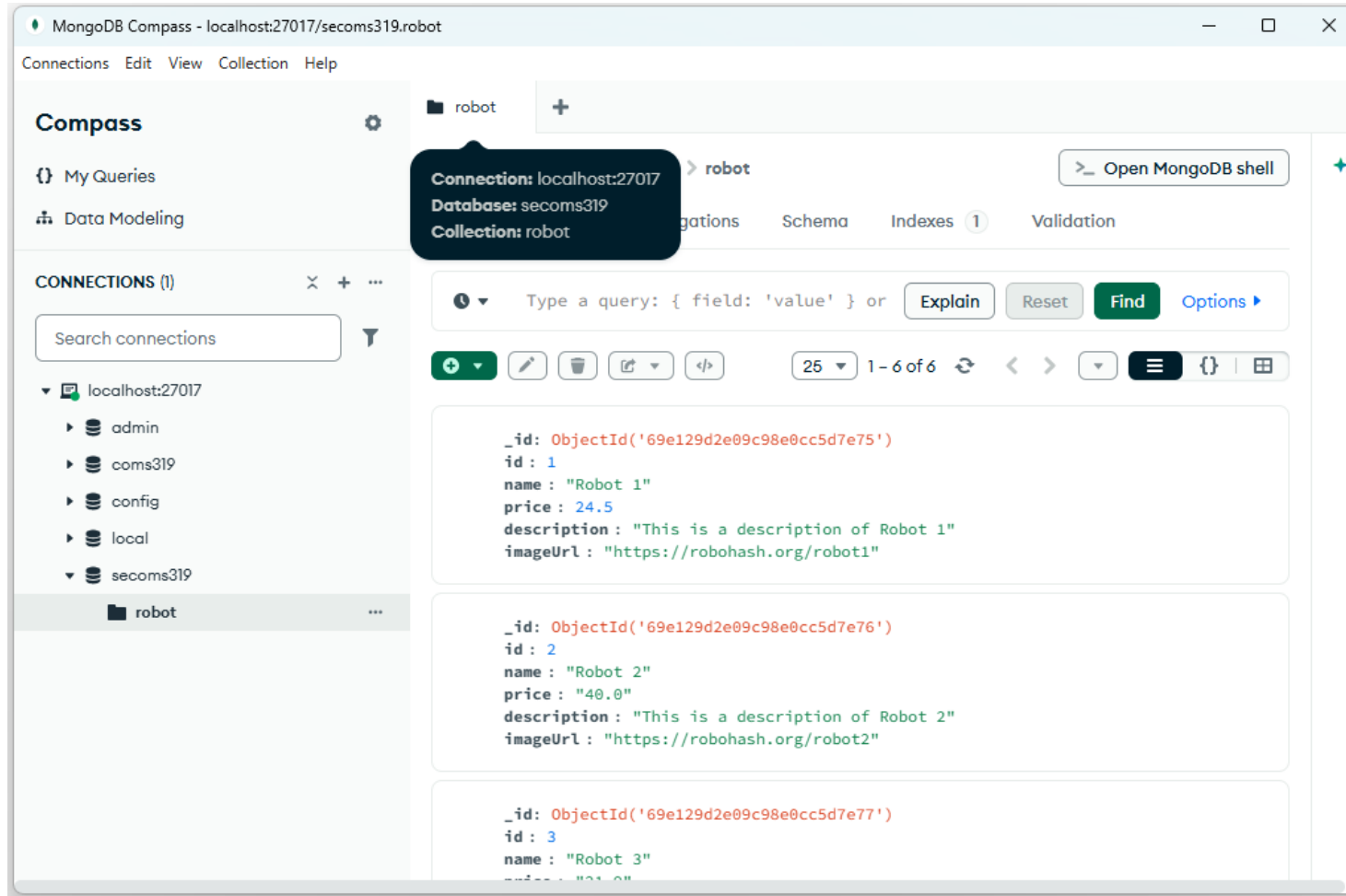






Create a new collection:
robot

Add the robots from `robot.json` manually :



```
[
  {
    "id": 1,
    "name": "Robot 1",
    "price": 24.5,
    "description": "This is a description of Robot 1",
    "imageUrl": "https://robohash.org/robot1"
  },
  {
    "id": 2,
    "name": "Robot 2",
    "price": 40.00,
    "description": "This is a description of Robot 2",
    "imageUrl": "https://robohash.org/robot2"
  },
  {
    "id": 3,
    "name": "Robot 3",
    "price": 20.50,
    "description": "This is a description of Robot 3",
    "imageUrl": "https://robohash.org/robot3"
  }
]
```

Reference:

<https://www.mongodb.com/languages/express-mongodb-rest-api-tutorial>

package.json

```
{
  "name": "backend",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC",
  "description": "",
  "dependencies": {
    "body-parser": "^2.2.0",
    "cors": "^2.8.5",
    "express": "^5.1.0",
    "mongodb": "^6.16.0"
  }
}
```

Make sure you have version 3.1.14 of nodemon

In the terminal enter the following:

```
nodemon -v // Expected 3.1.14
```

If you do not see the same version, then run:

Windows:

```
npm install -g nodemon@3.1.14
```

Mac:

If the previous command did not work, then run:

```
sudo npm install -g nodemon@3.1.14
```

In the backend folder, make a file named:
index.js

index.js

```
var express = require("express");
var cors = require("cors");
var app = express();
var fs = require("fs");
var bodyParser = require("body-parser");

app.use(cors());
app.use(bodyParser.json());

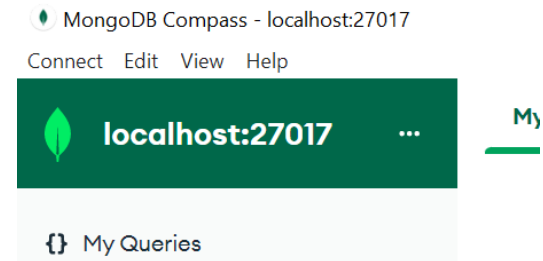
const port = "8080";
const host = "localhost";

//Add MongoDB
const { MongoClient } = require("mongodb");

//Add Constants
const url = "mongodb://127.0.0.1:27017";
const dbName = "secoms319";
const client = new MongoClient(url);
const db = client.db(dbName);

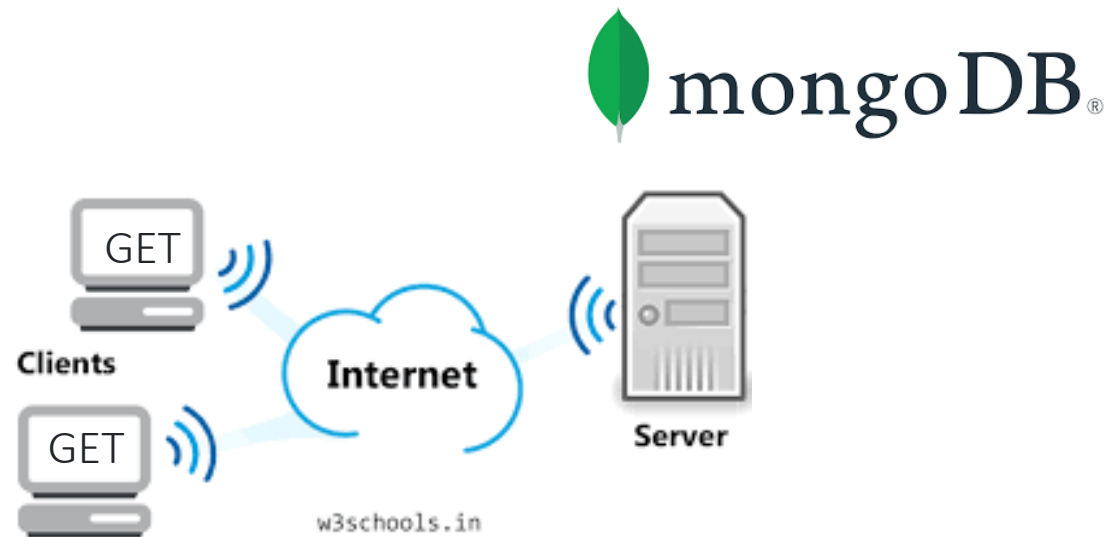
app.listen(port, () => {
  console.log("App listening at http://%s:%s",
    host, port);
});
```

Add mongodb:



← Add constants

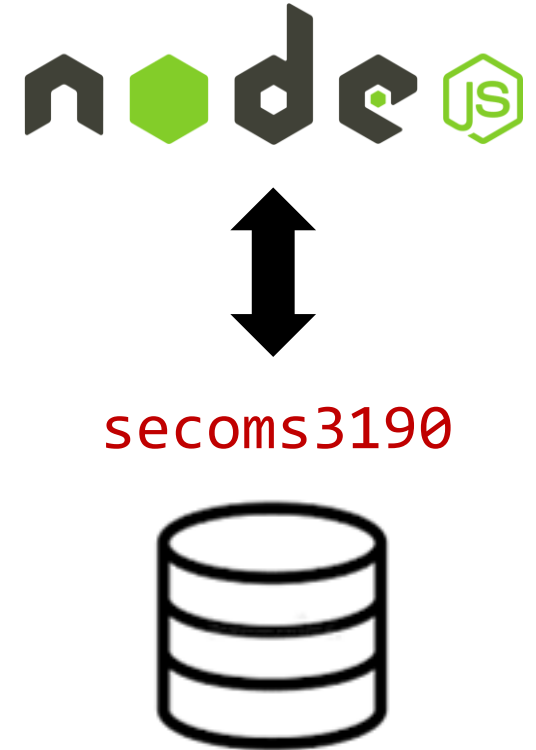
Request Method **GET**
`app.get("/listRobots ",`
add a new robot



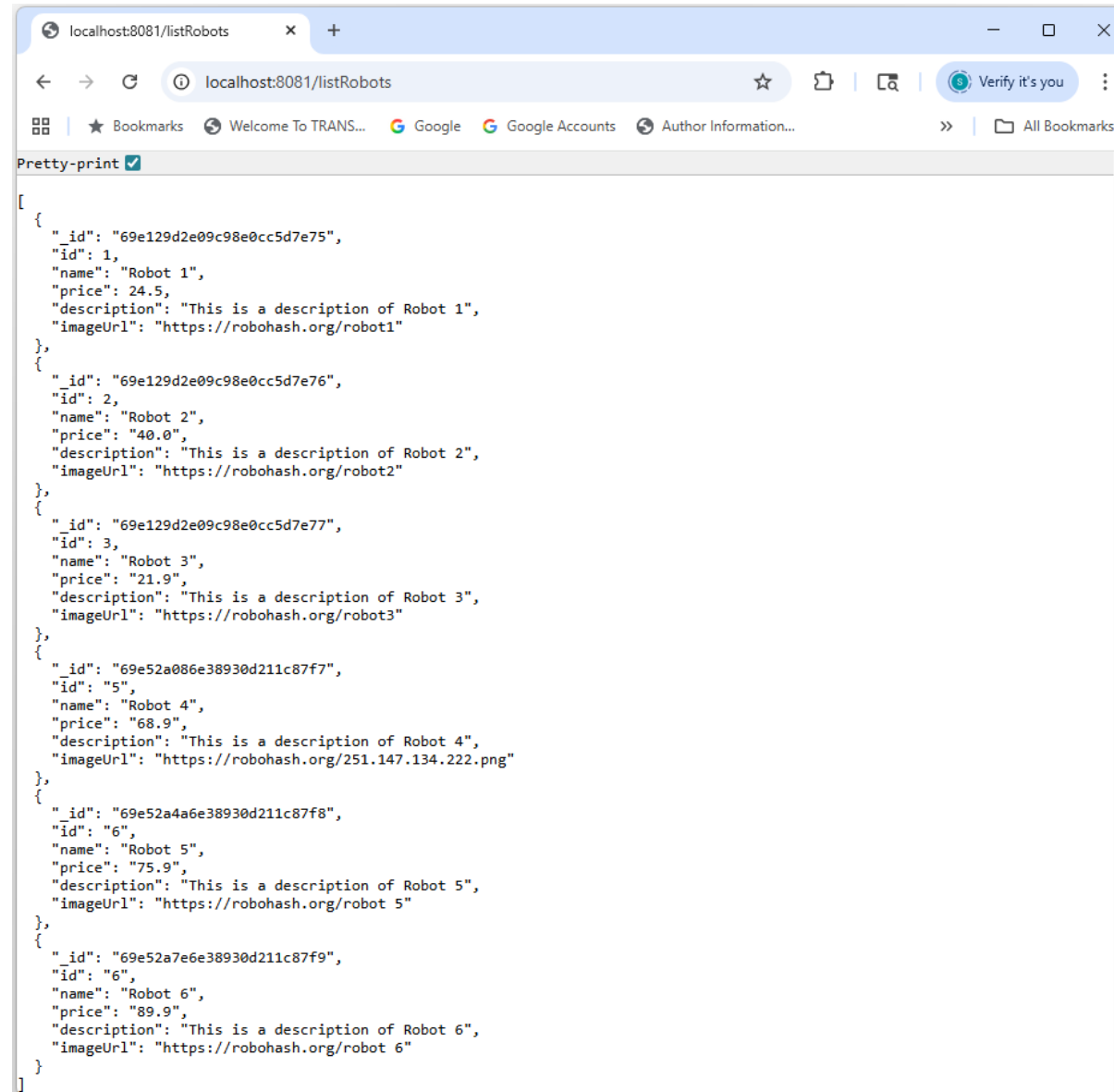
Add `get` Method using `/listRobots`:

```
app.get("/listRobots", async (req, res) => {  
  await client.connect();  
  console.log("Node connected successfully to GET  
MongoDB");  
  const query = {};  
  const results = await db  
    .collection("robot")  
    .find(query)  
    .limit(100)  
    .toArray();  
  console.log(results);  
  res.status(200);  
  res.send(results);  
});
```

What does this mean?
{ }



Test get request method : **http://localhost:8081/listRobots**



```
[{"_id": "69e129d2e09c98e0cc5d7e75", "id": 1, "name": "Robot 1", "price": 24.5, "description": "This is a description of Robot 1", "imageUrl": "https://robohash.org/robot1"}, {"_id": "69e129d2e09c98e0cc5d7e76", "id": 2, "name": "Robot 2", "price": 40.0, "description": "This is a description of Robot 2", "imageUrl": "https://robohash.org/robot2"}, {"_id": "69e129d2e09c98e0cc5d7e77", "id": 3, "name": "Robot 3", "price": 21.9, "description": "This is a description of Robot 3", "imageUrl": "https://robohash.org/robot3"}, {"_id": "69e52a086e38930d211c87f7", "id": "5", "name": "Robot 4", "price": 68.9, "description": "This is a description of Robot 4", "imageUrl": "https://robohash.org/251.147.134.222.png"}, {"_id": "69e52a4a6e38930d211c87f8", "id": "6", "name": "Robot 5", "price": 75.9, "description": "This is a description of Robot 5", "imageUrl": "https://robohash.org/robot 5"}, {"_id": "69e52a7e6e38930d211c87f9", "id": "6", "name": "Robot 6", "price": 89.9, "description": "This is a description of Robot 6", "imageUrl": "https://robohash.org/robot 6"}]
```

Postman

The screenshot displays the Postman application interface. The top bar includes navigation icons, a search bar, and buttons for 'Invite', 'Upgrade', and a notification bell. The left sidebar shows the 'My Workspace' section with a 'COLLECTIONS' list containing 'ahmed' and 'PUT sarfaraz'. The main panel shows a GET request to 'http://localhost:8081/listRobots' with a status of '[CONFLICT] GET sarfara'. The response is a 200 OK with a status of 88 ms and 1.57 KB. The response body is a JSON array of three robot objects, each with fields for _id, id, name, price, description, and imageUrl. The bottom bar includes 'Connect Git', 'Console', 'Terminal', and 'Globals'.

POST http://localhost:8080/ad [CONFLICT] GET sarfara +

ahmed > sarfaraz

GET http://localhost:8081/listRobots

Send

Docs Params Authorization Headers 10 Body Scripts Settings

Body Cookies Headers 8 Test Results

200 OK • 88 ms • 1.57 KB • Save Response

JSON Preview Visualize

```
1 [
2   {
3     "_id": "69e129d2e09c98e0cc5d7e75",
4     "id": 1,
5     "name": "Robot 1",
6     "price": "560000",
7     "description": "This is a description of Robot 1",
8     "imageUrl": "https://robohash.org/robot 1"
9   },
10  {
11    "_id": "69e129d2e09c98e0cc5d7e76",
12    "id": 2,
13    "name": "Robot 2",
14    "price": "40.0",
15    "description": "This is a description of Robot 2",
16    "imageUrl": "https://robohash.org/robot2"
17  },
18  {
19    "_id": "69e5aec53ffd5c1cd874442b",
20    "id": 3,
21    "name": "Robot 3",
22    "price": "10000",
23    "description": "This is a description of Robot 3",
24    "imageUrl": "https://robohash.org/robot 3"
25  },
26 ]
```

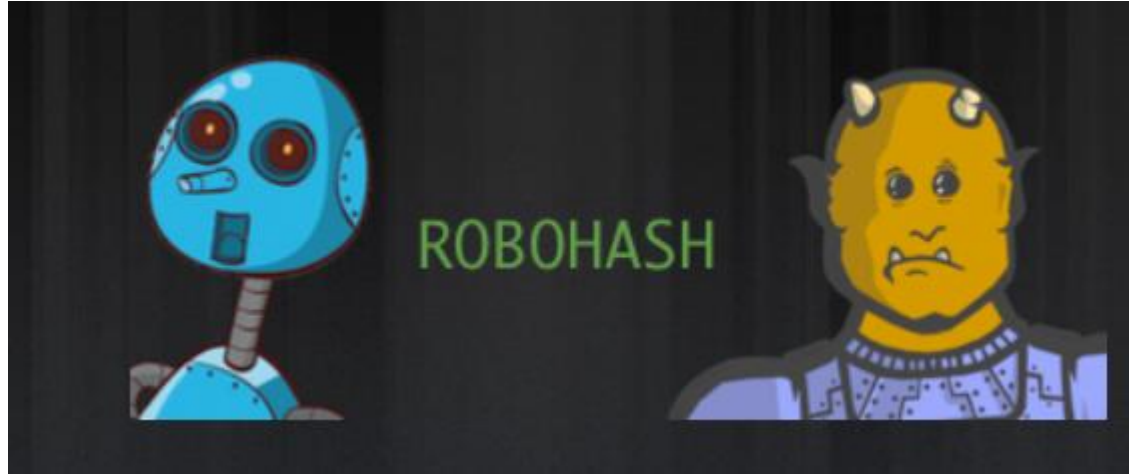
Connect Git Console Terminal

Globals Vault Tools

Request Method **POST**
`app.post("/addRobots",`
add a new robot



Goto to <https://robohash.org/> and generate a robot for you :



I generated this robot for my exercise:



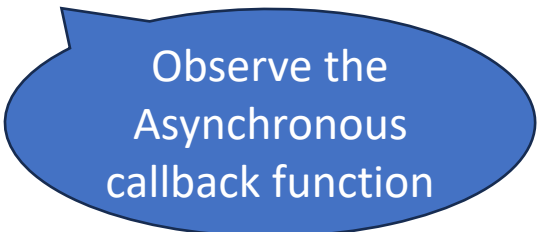
<https://robohash.org/Sarfaraz>

The minimum information required to add a new Robot is the next :

```
{  
  "id": 4,  
  "name": "Robot Sarfaraz",  
  "price":100.90,  
  "description": "I robot is one example of an image for my exercise",  
  "imageUrl": "https://robohash.org/Sarfaraz"  
}
```

Function post :

```
app.post("addRobots", async () => { });
```

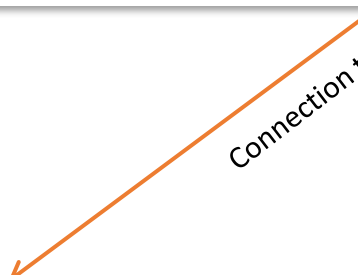


Observe the
Asynchronous
callback function

In the async callback
function ...

```
// MongoDB
const url = "mongodb://127.0.0.1:27017";
const dbName = "secoms319";
const client = new MongoClient(url);
const db = client.db(dbName);
```

Connection to DB



```
async (req, res) => {
```

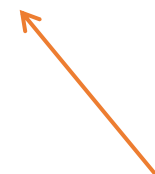
```
  await client.connect();
```

```
  const keys = Object.keys(req.body);
```

```
  const values = Object.values(req.body);
```

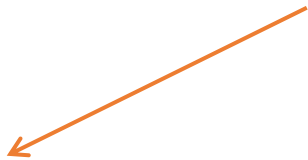
```
  ...
```

Object {
 key : value
}



In the async callback function ...

```
id : 4
name : "Robot Sarfaraz"
price : 100.9
description : "I robot is one example of an image for my exercise"
imageUrl : "https://robohash.org/Sarfaraz"
```



```
const newDocument = {
  "id": values[0],
  "name": values[1],
  "price": values[2],
  "description": values[3],
  "imageUrl": values[4]
};
```

```
console.log(newDocument);
```

```
// also "id": req.body.id,
// also "name": req.body.name,
// also "price": req.body.price,
// also "description": req.body.description,
// also "imageUrl": req.body.imageUrl
```

In the async callback function ...



```
const results = await db  
.collection("robot")  
.insertOne(newDocument);
```

```
res.status(200);  
res.send(results);
```



Add **post** Method using `"/addRobot"` :

```
// Add a new robot into the collection
app.post("/addRobots", async (req, res) => {
  try {
    await client.connect();
    const newDocument = {
      id: req.body.id,
      name: req.body.name,
      price: req.body.price,
      description: req.body.description,
      imageUrl: req.body.imageUrl,
    };
    const result = await db.collection("robot").insertOne(newDocument);
    res.status(200);
    res.send(result);
  } catch (error) {
    console.error("Could not add the new Robot" + error);
    res.status(500);
    res.send("Error adding new robot");
  } finally {
    await client.close();
  }
});
```

← Tries to connect to database

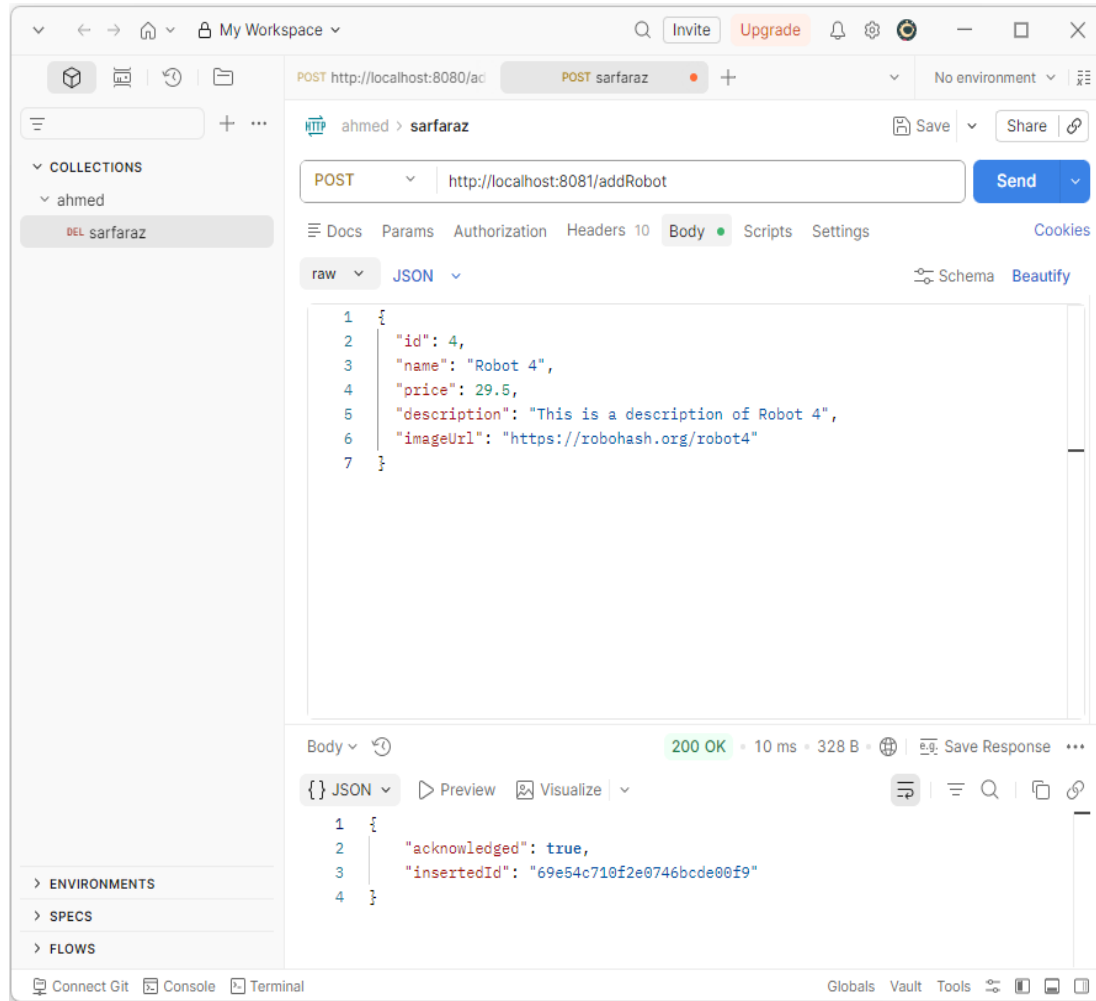
← If any error in trying to connect and send, error is caught

← If connected, and successfully added document or not, it is always good to close the connection after adding

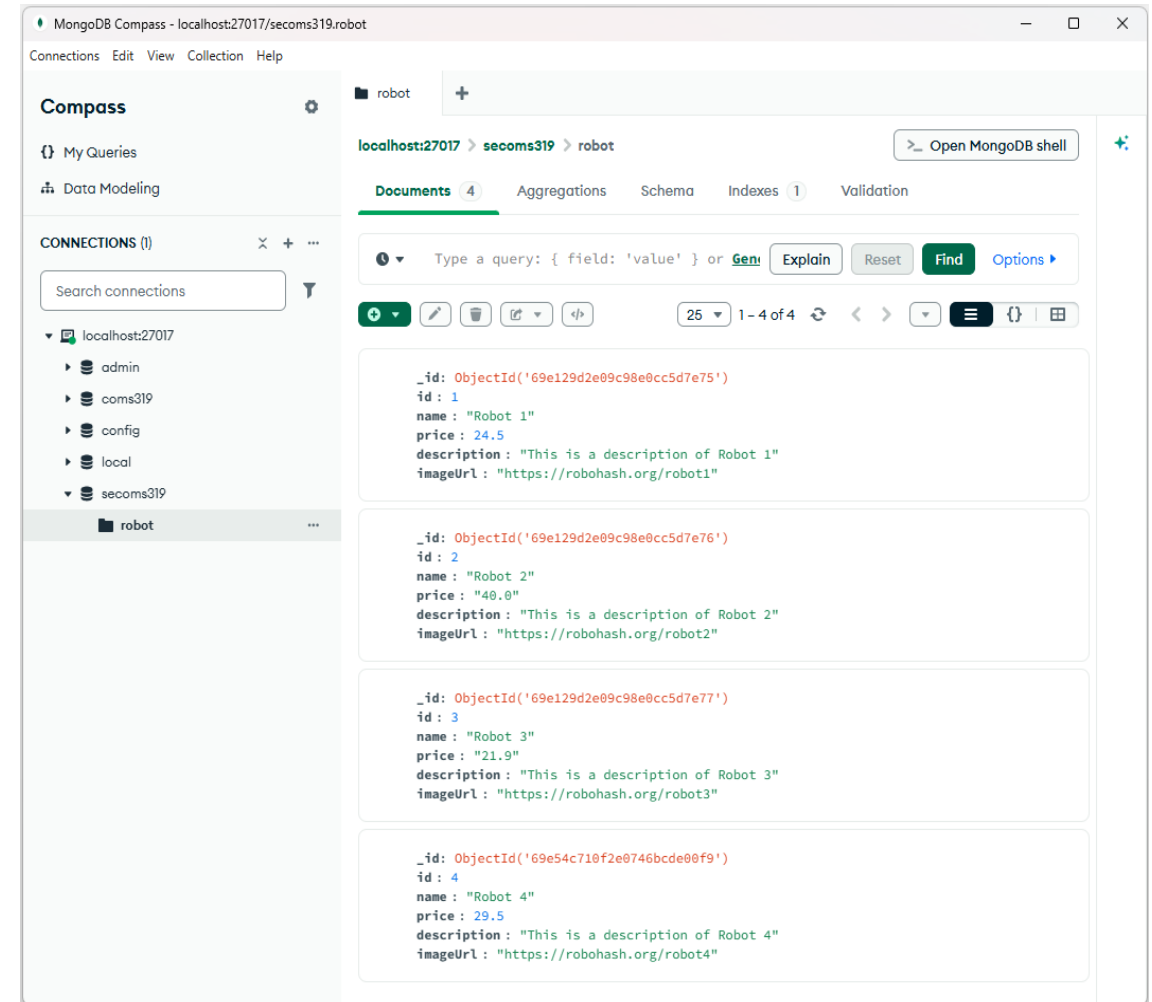
Let's suppose it finds an error from the server. The code is as follows :

```
app.post("/addRobots", async (req, res) => {  
  try {  
    .  
    .  
    .  
    res.status(200);  
    res.send(results);  
  
  } catch (error) {  
    console.error("An error occurred:", error);  
    res.status(500).send({ error: 'An internal server error occurred' });  
  }  
});
```

Postman



MongoDB



The above POST request method always responds with **200** HTTP status code of successful response.

Some other status codes that might be relevant for a POST request:

- **400 Bad Request:** This is used when the request is malformed, such as if the client sends data that doesn't validate.
- **401 Unauthorized:** If the operation requires authentication and the client has not provided it, this code is appropriate.
- **403 Forbidden:** If the client is authenticated but does not have permission to perform the requested operation.
- **409 Conflict:** This might be used if the operation would create a duplicate entry or some other conflict in the server's state.

Request Method UPDATE

```
app.put("/updateRobot/:id",  
        update a Robot by its id
```



Add **update** Method:

// Update Method -1

```
app.put("/updateRobot/:id", async (req, res) => {  
  const id = Number(req.params.id);  
  const query = { id: id };  
  await client.connect();  
  console.log("Robot to Update :", id);  
  // Data for updating the document, typically comes from the request body  
  console.log(req.body);  
  const updateData = {  
    $set: {  
      name: req.body.name,  
      price: req.body.price,  
      description: req.body.description,  
      imageUrl: req.body.imageUrl,  
    },  
  };  
  // Add options if needed, for example { upsert: true } to create a document if it  
  // doesn't exist  
  const options = {};  
  const results = await db  
    .collection("robot")  
    .updateOne(query, updateData, options);  
  res.status(200);  
  res.send(results);  
});
```

← Connect to MongoDB

← Put all that needs to be updated

← Tell where this updated data needs to be stored, by mentioning the name of the collection

← Pass 200 OK status

Add update Method

// Update Method -2

```
app.put("/updateRobot/:id", async (req, res) => {  
  try {  
    await client.connect();  
    const updateDoc = {  
      $set: req.body,  
    };  
  
    const result = await db  
      .collection("robots")  
      .updateOne({ id: Number(req.params.id) }, updateDoc);  
    if (result.modifiedCount === 0) {  
      return res.status(404).send({ error: "No robot found with that ID" });  
    }  
    res.status(200).send(result);  
  } catch (error) {  
    console.error("Error updating robot:", error);  
    res.status(500).send({ error: "An internal server error occurred" });  
  } finally {  
    await client.close();  
  }  
});
```

← Connect to MongoDB

← Put all that needs to be updated

← Tell where this updated data needs to be stored, by mentioning the name of the collection

← Pass the id that from url that needs to be modified and the updated data

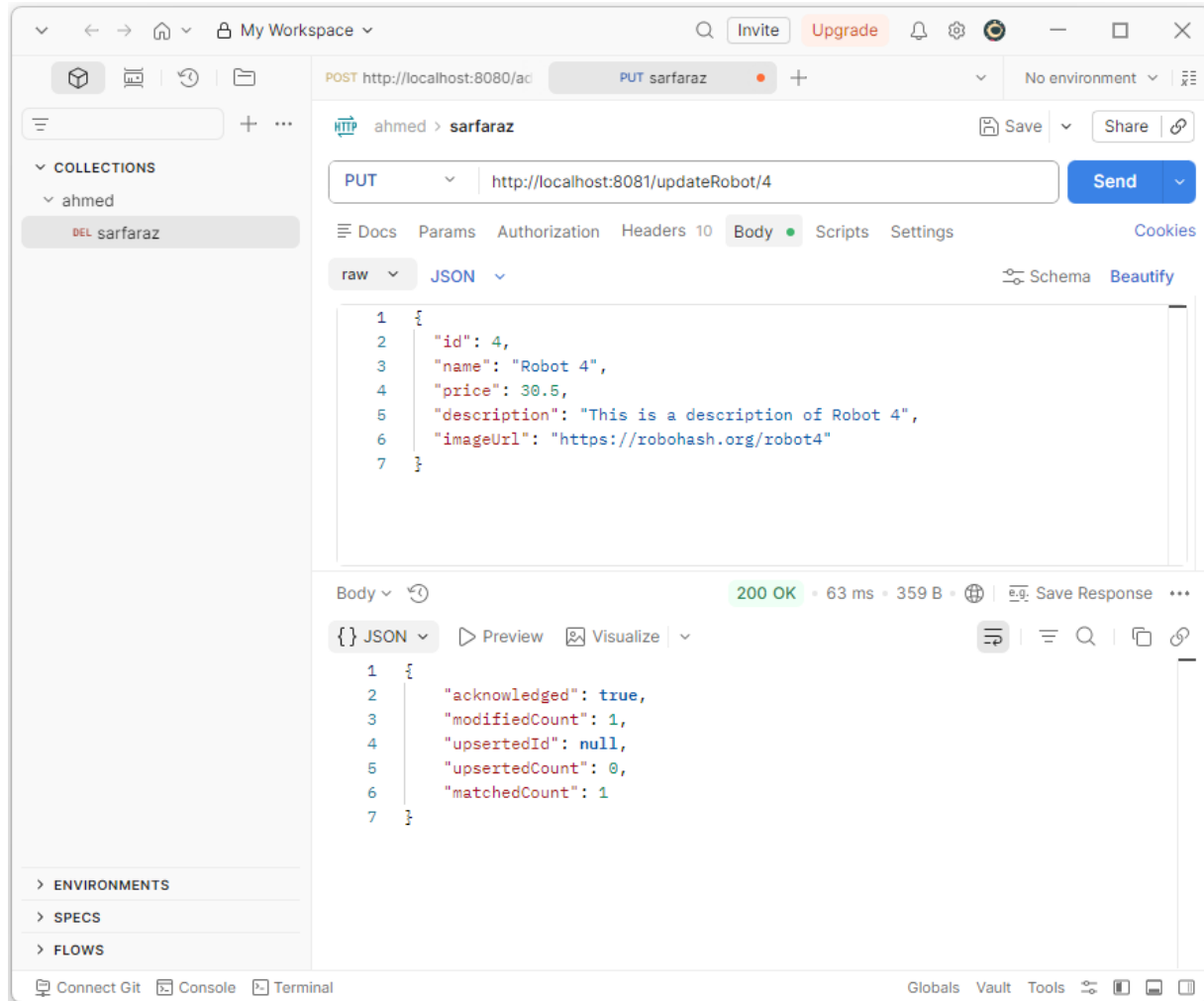
← Check if this worked, if not then throw a not found error, for No ID found.

← Otherwise, pass 200 OK status

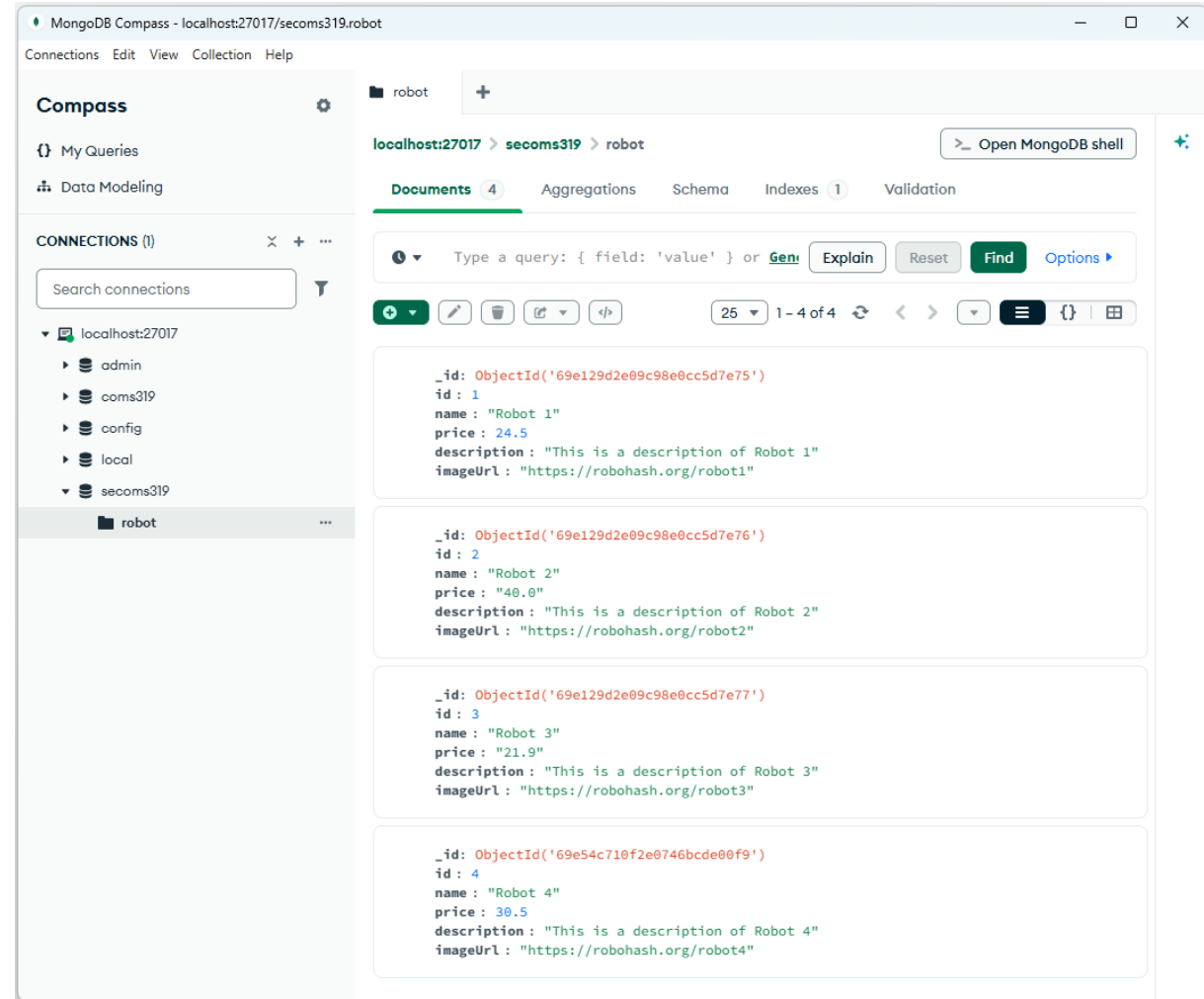
← Catch any other error

← Close the connection after update process is complete

Postman



MongoDB



Request Method Delete

```
app.delete("/deleteRobot/:id",  
delete a Robot by its id
```



Observe the collection `robot`:

```
_id: ObjectId('69e129d2e09c98e0cc5d7e77')  
id : 3  
name : "Robot 3"  
price : "21.9"  
description : "This is a description of Robot 3"  
imageUrl : "https://robohash.org/robot3"
```

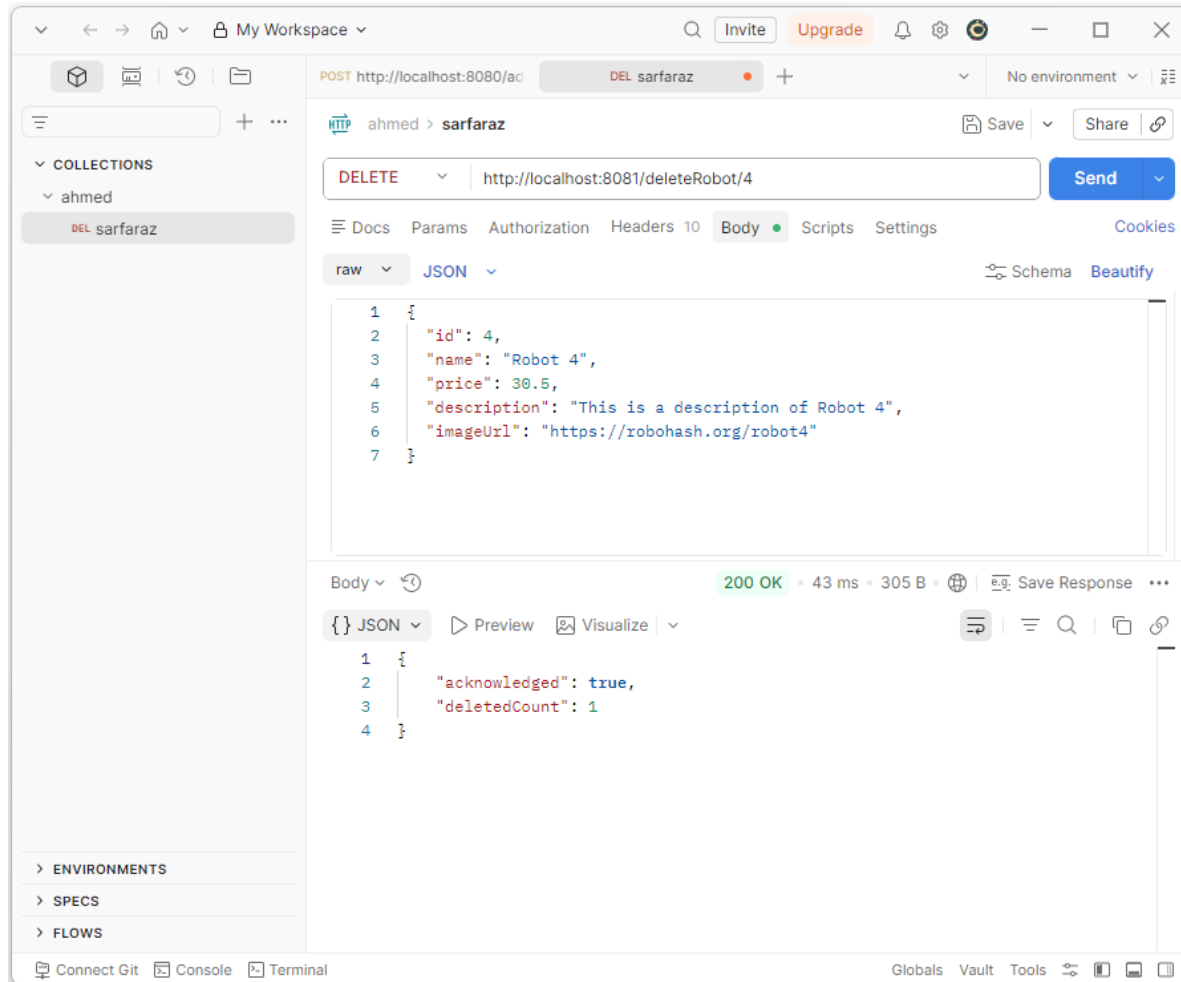
```
_id: ObjectId('69e54c710f2e0746bcde00f9')  
id : 4  
name : "Robot 4"  
price : 30.5  
description : "This is a description of Robot 4"  
imageUrl : "https://robohash.org/robot4"
```

Add delete Method using `"/deleteRobot/:id"`

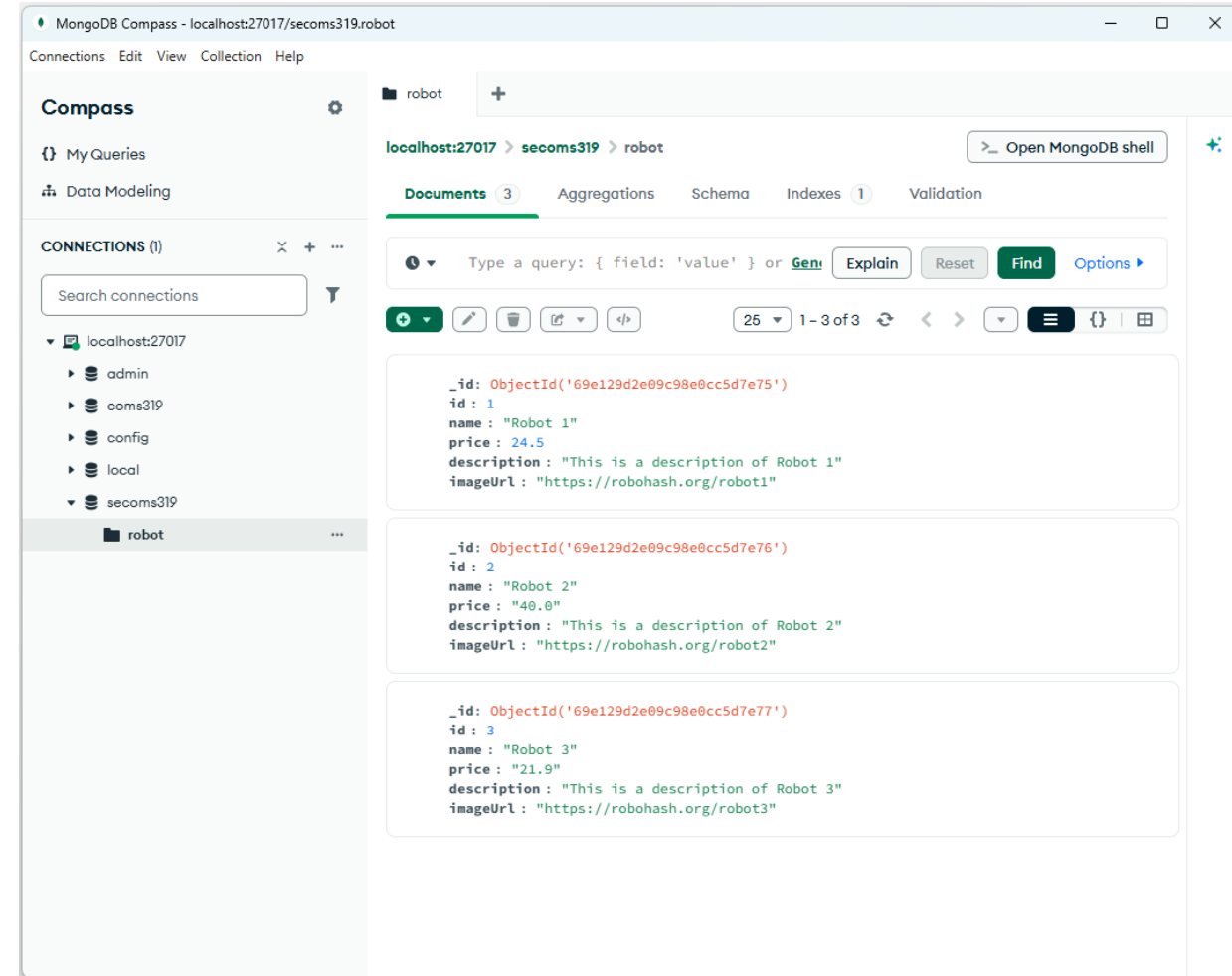
```
app.delete("/deleteRobot/:id", async (req, res) => {  
  try {  
    const id = Number(req.params.id);  
    await client.connect();  
    console.log("Robot to delete :", id);  
    const query = { id: id };  
    // read data from robot to delete to send it to frontend  
    const robotDeleted = await db.collection("robot").findOne(query);  
    if (robotDeleted) {  
      // delete  
      const results = await db.collection("robot").deleteOne(query);  
      res.status(200);  
      res.send(results);  
    } else {  
      res.status(404);  
      res.send("Robot Not Found");  
    }  
  } catch (error) {  
    console.error("Error deleting robot:", error);  
    res.status(500).send({ message: "Internal Server Error" });  
  }  
});
```

In this example we are using the **Robot Id** to reference it and delete it.

Postman



MongoDB



Actually, we use descriptive end points, such as: /addRobots, /updateRobot, /deleteRobot :

HTTP Method		CRUD action
POST	/addRobots	Create a new robot
GET	/listRobots	Read a list of robot
GET	/ {Id}	Read a single robot
PUT	/updateRobot/{id}	Update an existing robot
DELETE	/deleteRobot/{id}	Delete an existing robot

However, it is more appropriate as follows :

HTTP Method		CRUD action
POST	/robots	Create a new robot
GET	/robots	Read a list of robot
GET	/robots/{Id}	Read a single robot
PUT	/robots/{Id}	Update an existing robot
DELETE	/robots/{Id}	Delete an existing robot

Let us now connect this backend to frontend:

As you already know, we created a frontend project using the npm commands and now let's add a directory into the src named components and in that add the following files:

- Robots.jsx
- AddRobots.jsx
- PutRobots.jsx
- DeleteRobots.jsx

```
npm install bootstrap react-bootstrap
```

Robots.jsx

```
import React, { useState, useEffect } from "react";
import "bootstrap/dist/css/bootstrap.min.css";
import { Card, Row, Col, Container } from "react-bootstrap";
```

```
const Robots = () => {
  const [robots, setRobots] = useState([]);
```

```
  return (<>
    <div>
      <Container>
        <h1>Robots</h1>
        <Row
          xs={1}
          md={2}
          lg={3}
          className="g-4 d-flex flex-row align-items-stretch"
        >
          {robots.map((robot) => (
            <Col key={robot.id}>
              <Card>
                <Card.Img
                  variant="top"
                  src={robot.imageUrl}
                  style={{ height: "350px" }}
                  alt={robot.name}
                />
                <Card.Body>
                  <Card.Title style={{ fontSize: "18px" }}>
                    {robot.name}
                  </Card.Title>
                  <Card.Text style={{ color: "#000000" }}>
                    <strong>Price:</strong> ${robot.price}
                  </Card.Text>
                  <Card.Text style={{ color: "#000000" }}>
                    <strong>Cast:</strong>
                    {robot.description}
                  </Card.Text>
                </Card.Body>
              </Card>
            </Col>
          )))
        </Row>
      </Container>
    </div>
  </>
  );
};
export default Robots;
```

In JavaScript, we use the `fetch` function to retrieve data from a JSON file.

In React, the same concept applies, but all `fetch` calls are placed inside a `useEffect` hook.

This ensures that whenever the dependency array changes, `useEffect` automatically triggers and retrieves the latest information from the database.

Add these right before the **return** block:

```
useEffect(() => {  
  getRobots();  
}, []);
```

← This useEffect will call the getRobots function **every time** the component mounts.

```
function getRobots() {  
  fetch("http://127.0.0.1:8080/listRobots")  
    .then((response) => response.json())  
    .then((data) => {  
      setRobots(data);  
    })  
    .catch((error) => {  
      console.error("Error fetching Robots:",  
error);  
    });  
}
```

← Notice, we are using 127.0.0.1, which is the address for localhost

← We are setting the state variable robots once we get the data from the database

AddRobots.jsx

```
import React, { useState } from "react";

const AddRobots = () => {
  const [formData, setFormData] = useState({
    id: "",
    name: "",
    price: "",
    description: "",
    imageUrl: "",
  });

  const handleChange = (event) => {
    const { name, value } = event.target;
    setFormData((prevState) => ({
      ...prevState,
      [name]: value,
    }));
  };
};
```

```
return (
  <>
    <form onSubmit={handleSubmit}>
      <input type="text" name="id" value={formData.id}
        onChange={handleChange}
        placeholder="ID" required />{" "}
      <br /><br />
      <input type="text" name="name" value={formData.name}
        onChange={handleChange}
        placeholder="Robot Name" required />
      <br /><br />
      <input type="text" name="price"
        value={formData.price}
        onChange={handleChange}
        placeholder="Robot Price" required />
      <br /><br />
      <input type="text" name="description"
        value={formData.description}
        onChange={handleChange}
        placeholder="Robot Description" required />
      <br /> <br />
      <textarea name="imageUrl"
        value={formData.imageUrl}
        onChange={handleChange}
        placeholder="Robot Image URL" required
      /> <br /><br />
      <button type="submit">Add Robot</button>
    </form>
  </>
);};

export default AddRobots;
```

Add these right before the **return** block:

```
const handleSubmit = async (event) => {
  event.preventDefault();
  const url = "http://127.0.0.1:8080/addRobots";
  try {
    const response = await fetch(url, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify(formData),
    });

    if (!response.ok) {
      throw new Error("Failed to add new Robot");
    }
    const result = await response.json();
    console.log("Success:", result);
    alert("Robot added successfully!");
    // Optionally reset form or navigate away
    setFormData({
      id: "",
      name: "",
      price: "",
      description: "",
      imageUrl: "",
    });
  } catch (error) {
    console.error("Error:", error);
    alert("Error adding Robot");
  }
};
```



Notice, we are using **127.0.0.1**, which is the address for localhost



Since MongoDB uses JSON objects to store data, we need to specify how we are passing our data.



We are setting the formData as the body for the request



Once the robot is successfully added, we reset the form.

Lastly update the **App.jsx** to render the component upon the button click:

```
import React, { useState } from "react";
import Robots from "../components/Robots";
import AddRobots from "../components/AddRobots";
import UpdateRobot from "../components/UpdateRobot";
import DeleteRobot from "../components/DeleteRobot";
import { Container, Row, Col, ButtonGroup, Button } from "react-bootstrap";

const App = () => {
  const [activeComponent, setActiveComponent] = useState("robots");

  return (
    <Container className="mt-4">
      <Row className="mb-3">
        <Col>
          <ButtonGroup>
            <Button
              variant={
                activeComponent === "robots" ? "primary" :
"outline-primary"
              }
              onClick={() => setActiveComponent("robots")}
            >
              View Robots
            </Button>
          </ButtonGroup>
        </Col>
      </Row>
    </Container>
  );
};
```

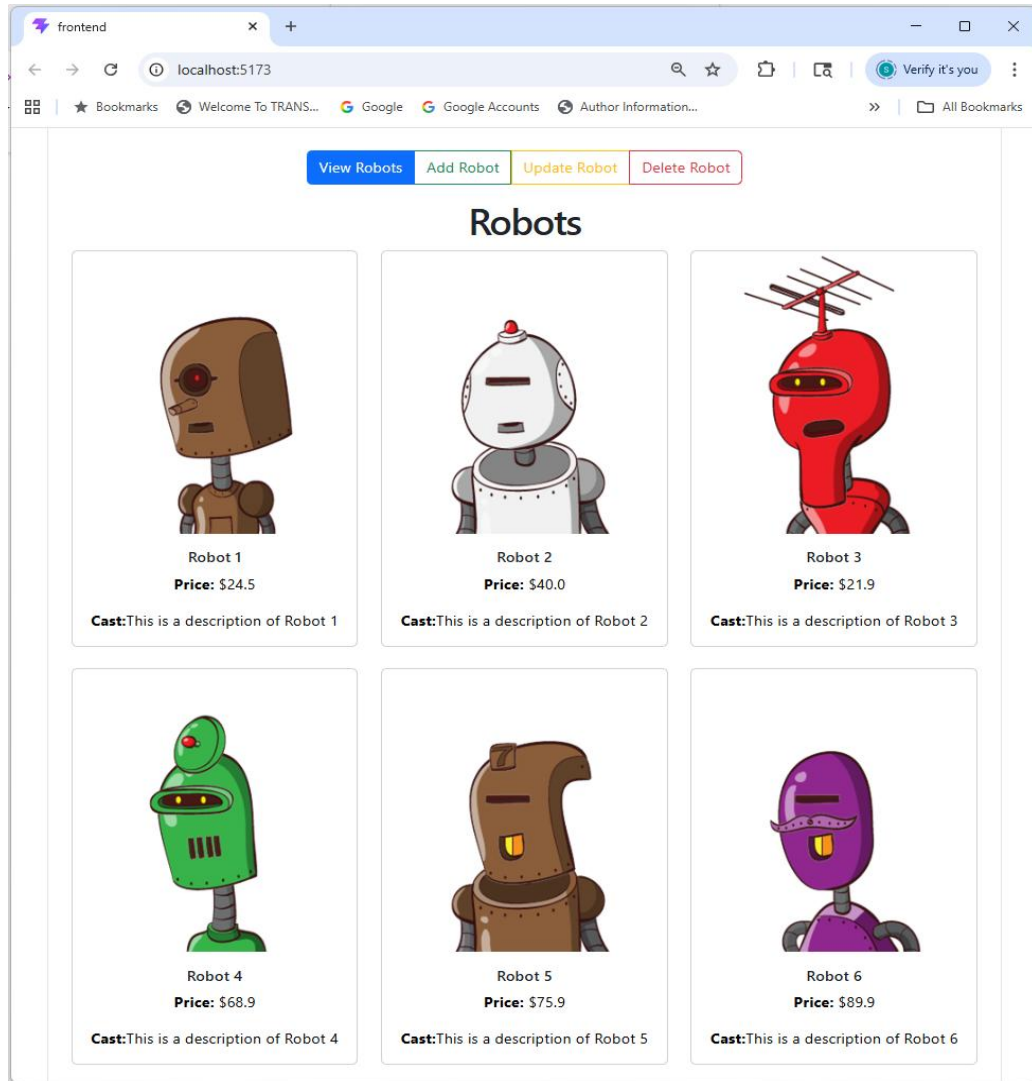
```
<Button
  variant={
    activeComponent === "add" ?
"success" : "outline-success"
  }
  onClick={() =>
    setActiveComponent("add")
  }
>
  Add Robot
</Button>
<Button
  variant={
    activeComponent === "update" ?
"warning" : "outline-warning"
  }
  onClick={() =>
    setActiveComponent("update")
  }
>
  Update Robot
</Button>
<Button
  variant={
    activeComponent === "delete" ?
"danger" : "outline-danger"
  }
  onClick={() =>
    setActiveComponent("delete")
  }
>
  Delete Robot
</Button>
</ButtonGroup>
</Col>
</Row>
```

```
<Row>
  <Col>
    {activeComponent === "robots" &&
      <Robots />
    {activeComponent === "add" &&
      <AddRobots />
    {activeComponent === "update" &&
      <UpdateRobot />
    {activeComponent === "delete" &&
      <DeleteRobot />
    }
  </Col>
</Row>
</Container>
);
};

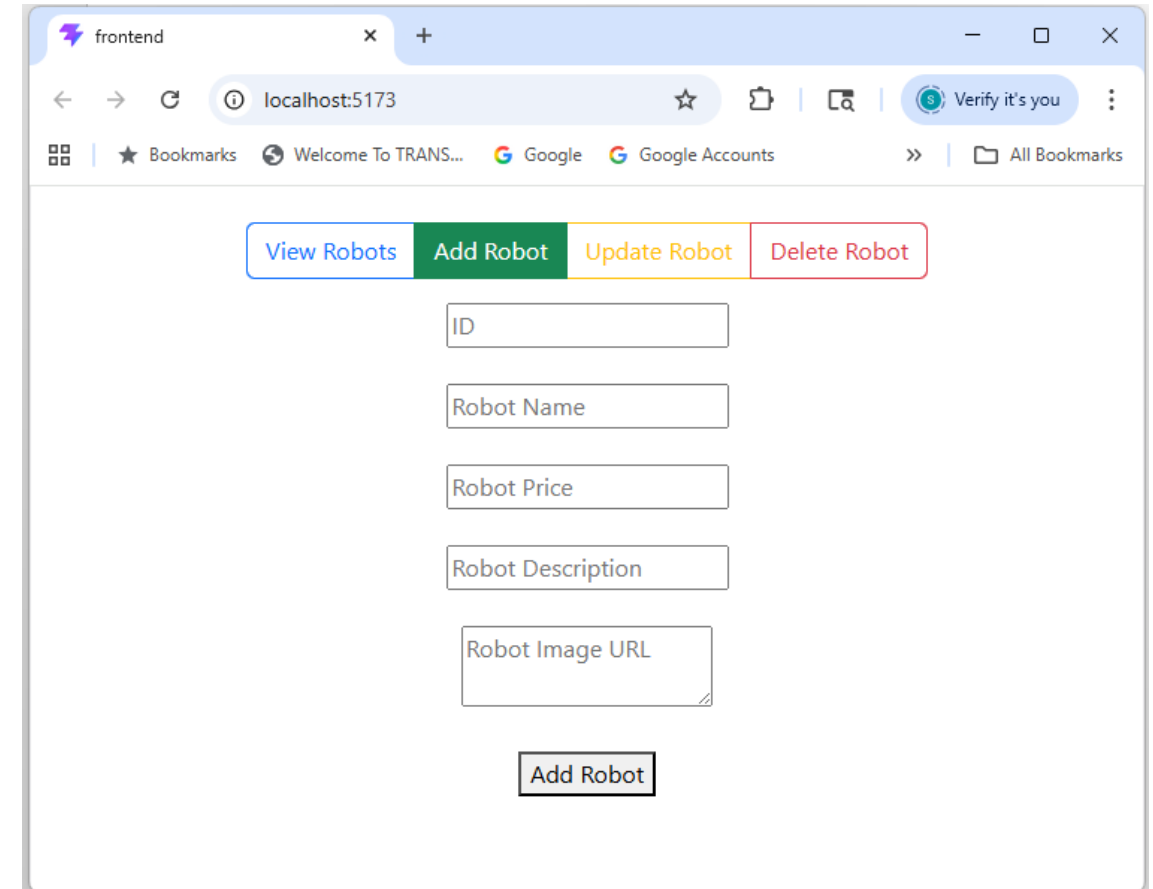
export default App;
```

With that we are done with the Activity. Here is how it looks!

GET

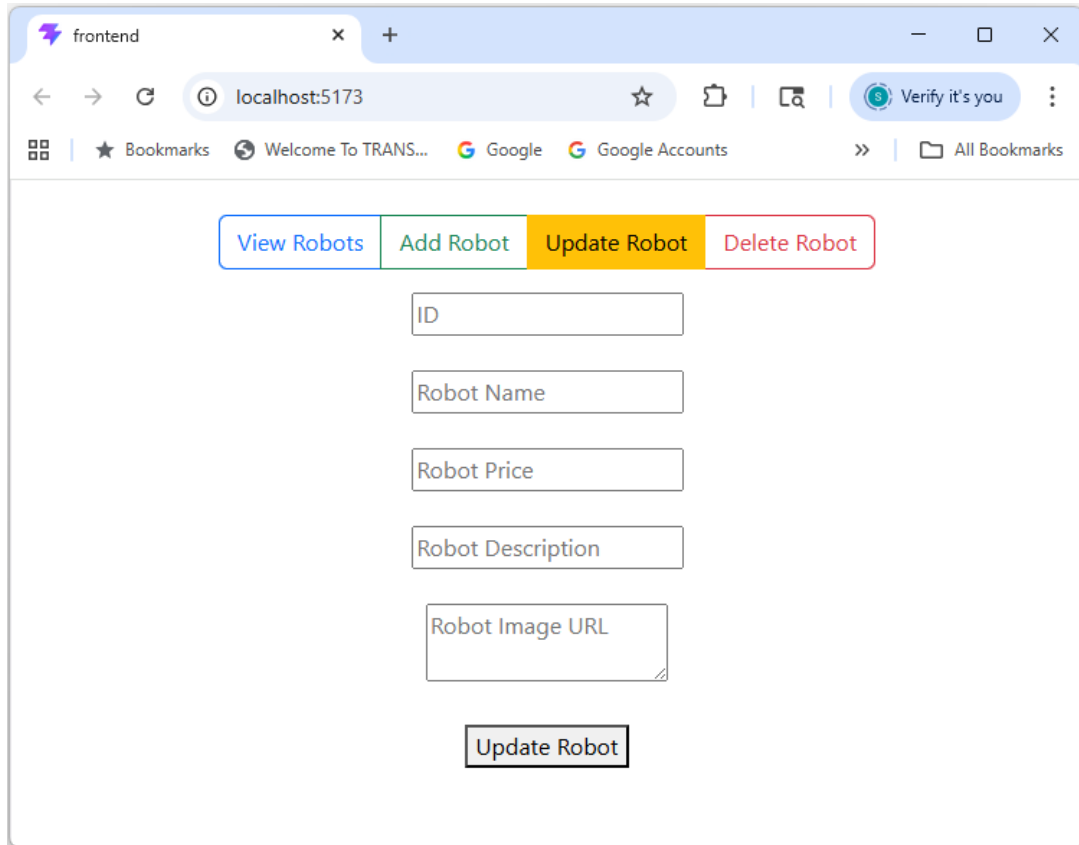


POST



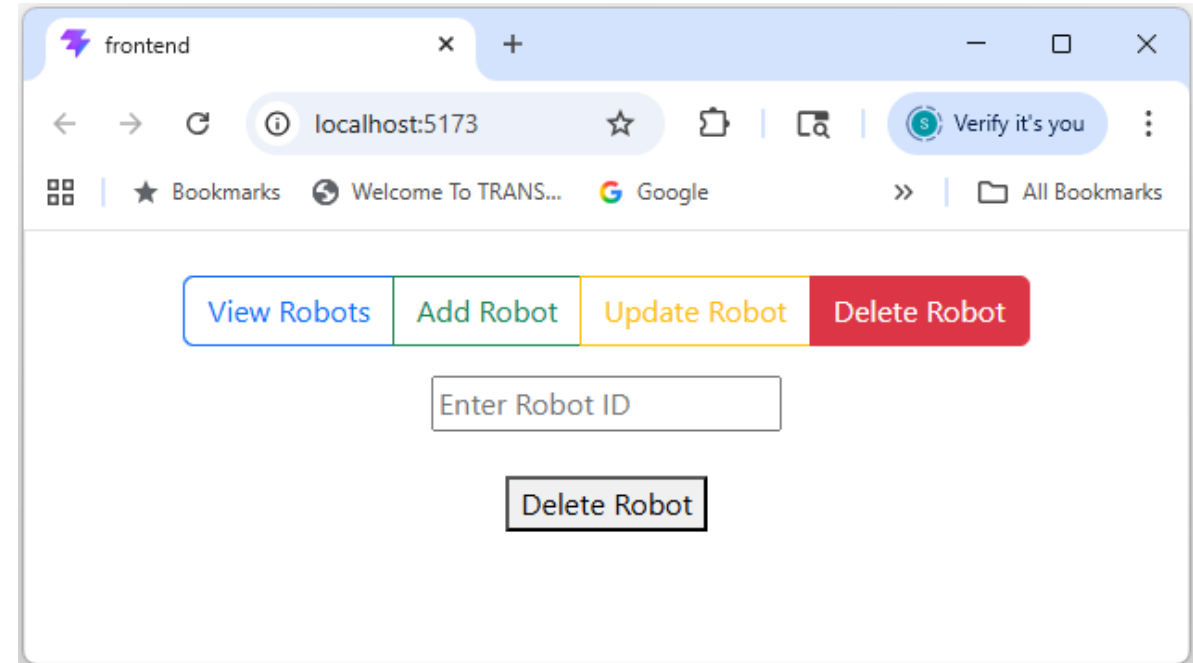
The next part of the Activity (PUT, DELETE) will be discussed in the next lecture!

PUT



A screenshot of a web browser window showing the PUT endpoint of a robot management application. The browser's address bar displays 'localhost:5173'. The application interface includes a navigation bar with four buttons: 'View Robots' (blue), 'Add Robot' (green), 'Update Robot' (yellow), and 'Delete Robot' (red). Below the navigation bar, there are five input fields stacked vertically: 'ID', 'Robot Name', 'Robot Price', 'Robot Description', and 'Robot Image URL'. At the bottom of the form, there is a single 'Update Robot' button.

DELETE



A screenshot of a web browser window showing the DELETE endpoint of the robot management application. The browser's address bar displays 'localhost:5173'. The application interface features the same navigation bar as the PUT endpoint, with buttons for 'View Robots', 'Add Robot', 'Update Robot', and 'Delete Robot'. Below the navigation bar, there is a single input field labeled 'Enter Robot ID'. At the bottom of the form, there is a single 'Delete Robot' button.